TRABAJO PRÁCTICO GRUPAL: Consultas

Avanzadas con JPQL Objetivo

Aprender a consultar la base de datos relacional orientada a objetos utilizando el lenguaje de consultas de JPA (JPQL), aplicando filtros, agregaciones, funciones de agrupamiento, combinaciones entre tablas (JOINs) y subconsultas correlacionadas.

Instrucciones Generales

1. Utilizar el método em.createQuery(jpql, Clasedestino.class) de JPA. 2. Pasar los parámetros siempre de forma nombrada (ej. :parametro) para evitar vulnerabilidades de inyección SQL.
3. Todas las consultas deben basarse en los nombres de las **entidades y atributos Java**, no en los nombres de las tablas de la base de datos.

Nivel 1: Consultas Básicas y Proyecciones

1. Consulta de Entidades Completas

- **Consigna:** Obtener la lista completa de todas las facturas de venta registradas en el sistema.

Consulta:

```java
private static void consulta01FacturasCompletas(EntityManager em) {
    String jpql = "SELECT fv FROM FacturaVenta fv";
    imprimirConsulta("1. Consulta de entidades completas", jpql);

    List<FacturaVenta> facturas = em.createQuery(jpql, FacturaVenta.class).getResultList();
    imprimirFacturas(facturas);
}
```

Consola:

```
1. Consulta de entidades completas
JPQL:
SELECT fv FROM FacturaVenta fv
Resultados:
id=3, numero=1001, fechaEmision=2026-09-02, estado=EMITIDA, importeTotal=75000.0
id=4, numero=1002, fechaEmision=2026-09-03, estado=EMITIDA, importeTotal=25000.0
id=5, numero=1003, fechaEmision=2026-09-04, estado=EMITIDA, importeTotal=8000.0
id=6, numero=1004, fechaEmision=2026-09-05, estado=ANULADA, importeTotal=12500.0
id=7, numero=1005, fechaEmision=2026-09-06, estado=PENDIENTE, importeTotal=45000.0
id=8, numero=1006, fechaEmision=2026-09-07, estado=EMITIDA, importeTotal=90000.0
id=9, numero=1007, fechaEmision=2026-09-08, estado=EMITIDA, importeTotal=16500.0
```

2. Proyección de Atributos Específicos

- **Consigna:** Seleccionar únicamente el número de factura, la fecha de emisión y el importe total de todas las facturas de venta.

Consulta:



```java
private static void consulta02ProyeccionFacturas(EntityManager em) {
        String jpql = "SELECT fv.numero, fv.fechaEmision, fv.importeTotal FROM FacturaVenta fv";
        imprimirConsulta("2. Proyeccion de atributos especificos", jpql);

        List<Object[]> filas = em.createQuery(jpql, Object[].class).getResultList();
        for (Object[] fila : filas) {
            System.out.println("numero=" + fila[0] + ", fechaEmision=" + fila[1] + ", importeTotal=" + fila[2]);
        }
        imprimirSinResultados(filas);
    }
```

Consola:

```
2. Proyeccion de atributos especificos
JPQL:
SELECT fv.numero, fv.fechaEmision, fv.importeTotal FROM FacturaVenta fv
Resultados:
numero=1001, fechaEmision=2026-09-02, importeTotal=75000.0
numero=1002, fechaEmision=2026-09-03, importeTotal=25000.0
numero=1003, fechaEmision=2026-09-04, importeTotal=8000.0
numero=1004, fechaEmision=2026-09-05, importeTotal=12500.0
numero=1005, fechaEmision=2026-09-06, importeTotal=45000.0
numero=1006, fechaEmision=2026-09-07, importeTotal=90000.0
numero=1007, fechaEmision=2026-09-08, importeTotal=16500.0
```

3. Filtrado por Igualdad (WHERE)

- **Consigna:** Obtener todos los artículos que pertenecen a un rubro con una denominación específica (ej. "Electrónica").

Consulta:

```java
private static void consulta03ArticulosPorRubro(EntityManager em, String denominacionRubro) {
        String jpql = "SELECT a FROM Articulo a WHERE a.rubro.denominacion = :denominacionRubro";
        imprimirConsulta("3. Filtrado por igualdad", jpql);

        List<Articulo> articulos = em.createQuery(jpql, Articulo.class)
                .setParameter("denominacionRubro", denominacionRubro)
                .getResultList();
        imprimirArticulos(articulos);
    }
```

Consola:

```
3. Filtrado por igualdad
JPQL:
SELECT a FROM Articulo a WHERE a.rubro.denominacion = :denominacionRubro
Resultados:
id=3, codigo=ART01, denominacion=Teclado Mecanico, rubro=Electronica, marca=Logitech
id=4, codigo=ART02, denominacion=Mouse Inalambrico, rubro=Electronica, marca=Logitech
id=5, codigo=ART03, denominacion=Monitor 24 pulgadas, rubro=Electronica, marca=Samsung
id=6, codigo=ART04, denominacion=Cable HDMI, rubro=Electronica, marca=Generica
```

4. Filtrado por Rango de Fechas (BETWEEN)

- **Consigna:** Listar todas las facturas de venta emitidas dentro de un rango de fechas determinado.

Consulta:

```java
private static void consulta04FacturasPorRangoFechas(EntityManager em, Date fechaDesde, Date fechaHasta) {
        String jpql = "SELECT fv FROM FacturaVenta fv WHERE fv.fechaEmision BETWEEN :fechaDesde AND :fechaHasta";
        imprimirConsulta("4. Filtrado por rango de fechas", jpql);

        List<FacturaVenta> facturas = em.createQuery(jpql, FacturaVenta.class)
                .setParameter("fechaDesde", fechaDesde)
                .setParameter("fechaHasta", fechaHasta)
                .getResultList();
        imprimirFacturas(facturas);
    }
```

Consola:

```
4. Filtrado por rango de fechas
JPQL:
SELECT fv FROM FacturaVenta fv WHERE fv.fechaEmision BETWEEN :fechaDesde AND :fechaHasta
Resultados:
id=3, numero=1001, fechaEmision=2026-09-02, estado=EMITIDA, importeTotal=75000.0
id=4, numero=1002, fechaEmision=2026-09-03, estado=EMITIDA, importeTotal=25000.0
id=5, numero=1003, fechaEmision=2026-09-04, estado=EMITIDA, importeTotal=8000.0
id=6, numero=1004, fechaEmision=2026-09-05, estado=ANULADA, importeTotal=12500.0
id=7, numero=1005, fechaEmision=2026-09-06, estado=PENDIENTE, importeTotal=45000.0
id=8, numero=1006, fechaEmision=2026-09-07, estado=EMITIDA, importeTotal=90000.0
id=9, numero=1007, fechaEmision=2026-09-08, estado=EMITIDA, importeTotal=16500.0
```

Nivel 2: Condicionales Combinados, Operadores de Texto y Agregaciones Básicas

5. Condicionales Complejos y Verificación de Nulos (AND, OR, IS NULL)

- **Consigna:** Obtener las facturas cuyo estado sea "EMITIDA", con un importe total superior a $10,000 y que no hayan sido anuladas (fechaAnulacion sea nula).

Consulta:

```java
private static void consulta05FacturasEmitidasNoAnuladas(EntityManager em) {
        String jpql = """
                SELECT fv
                FROM FacturaVenta fv
                WHERE fv.estado = :estado
                  AND fv.importeTotal > :importeMinimo
                  AND fv.fechaAnulacion IS NULL
                """;
        imprimirConsulta("5. Condicionales complejos e IS NULL", jpql);

        List<FacturaVenta> facturas = em.createQuery(jpql, FacturaVenta.class)
                .setParameter("estado", "EMITIDA")
                .setParameter("importeMinimo", 10000.0)
                .getResultList();
        imprimirFacturas(facturas);
    }
```

Consola:

```
5. Condicionales complejos e IS NULL
JPQL:
SELECT fv
FROM FacturaVenta fv
WHERE fv.estado = :estado
  AND fv.importeTotal > :importeMinimo
  AND fv.fechaAnulacion IS NULL
Resultados:
id=3, numero=1001, fechaEmision=2026-09-02, estado=EMITIDA, importeTotal=75000.0
id=4, numero=1002, fechaEmision=2026-09-03, estado=EMITIDA, importeTotal=25000.0
id=8, numero=1006, fechaEmision=2026-09-07, estado=EMITIDA, importeTotal=90000.0
id=9, numero=1007, fechaEmision=2026-09-08, estado=EMITIDA, importeTotal=16500.0
```

6. Búsqueda por Patrón de Texto (LIKE y LOWER)

- **Consigna:** Buscar todos los clientes cuya denominación contenga un texto parcial (sin importar mayúsculas/minúsculas) o cuyo CUIT/CUIL comience con "20-".

Consulta:

```java
private static void consulta06ClientesPorTextoOCuit(EntityManager em, String textoParcial, String prefijoCuit) {
        String jpql = """
                SELECT c
                FROM Cliente c
                WHERE LOWER(c.denominacion) LIKE LOWER(:textoParcial)
                   OR c.cuitCuil LIKE :prefijoCuit
                """;
        imprimirConsulta("6. Busqueda por patron de texto", jpql);

        List<Cliente> clientes = em.createQuery(jpql, Cliente.class)
                .setParameter("textoParcial", "%" + textoParcial + "%")
                .setParameter("prefijoCuit", prefijoCuit + "%")
                .getResultList();
        imprimirClientes(clientes);
    }
```

Consola:

```
6. Busqueda por patron de texto
JPQL:
SELECT c
FROM Cliente c
WHERE LOWER(c.denominacion) LIKE LOWER(:textoParcial)
   OR c.cuitCuil LIKE :prefijoCuit
Resultados:
id=2, denominacion=Cliente Mayorista, cuitCuil=20-12345678-9
id=3, denominacion=Cliente Minorista, cuitCuil=27-87654321-0
```

7. Valores Distintos y Ordenamiento (DISTINCT y ORDER BY)

- **Consigna:** Obtener sin duplicados todos los estados posibles registrados en las facturas de venta, ordenados alfabéticamente de forma ascendente.

Consulta:

```java
private static void consulta07EstadosDistintos(EntityManager em) {
        String jpql = "SELECT DISTINCT fv.estado FROM FacturaVenta fv ORDER BY fv.estado ASC";
        imprimirConsulta("7. Valores distintos y ordenamiento", jpql);

        List<String> estados = em.createQuery(jpql, String.class).getResultList();
        for (String estado : estados) {
            System.out.println("estado=" + estado);
        }
        imprimirSinResultados(estados);
    }
```

Consola:

```
7. Valores distintos y ordenamiento
JPQL:
SELECT DISTINCT fv.estado FROM FacturaVenta fv ORDER BY fv.estado ASC
Resultados:
estado=ANULADA
estado=EMITIDA
estado=PENDIENTE
```

8. Funciones de Agregación Simples (COUNT, SUM, AVG)

- **Consigna:** Obtener la cantidad total de facturas emitidas, la suma acumulada de sus importes totales y el importe promedio devuelto en un solo objeto/arreglo.

Consulta:

```java
private static void consulta08AgregacionesFacturas(EntityManager em) {
        String jpql = "SELECT COUNT(fv), SUM(fv.importeTotal), AVG(fv.importeTotal) FROM FacturaVenta fv";
        imprimirConsulta("8. Funciones de agregacion simples", jpql);

        Object[] fila = em.createQuery(jpql, Object[].class).getSingleResult();
        System.out.println("cantidad=" + fila[0] + ", sumaImportes=" + fila[1] + ", promedioImportes=" + fila[2]);
    }
```

Consola:

```
8. Funciones de agregacion simples
JPQL:
SELECT COUNT(fv), SUM(fv.importeTotal), AVG(fv.importeTotal) FROM FacturaVenta fv
Resultados:
cantidad=7, sumaImportes=272000.0, promedioImportes=38857.142857142855
```

9. Operador de Inclusión (IN)

- **Consigna:** Obtener todos los puntos de venta cuyo número coincida con una lista de enteros proporcionada por parámetro (ej. 1, 2, 5).

Consulta:



```java
private static void consulta09PuntosVentaPorNumeros(EntityManager em, List<Integer> numeros) {
        String jpql = "SELECT pv FROM PuntoVenta pv WHERE pv.numero IN :numeros";
        imprimirConsulta("9. Operador de inclusion IN", jpql);

        List<PuntoVenta> puntosVenta = em.createQuery(jpql, PuntoVenta.class)
                .setParameter("numeros", numeros)
                .getResultList();
        imprimirPuntosVenta(puntosVenta);
    }
```

Consola:

```
9. Operador de inclusion IN
JPQL:
SELECT pv FROM PuntoVenta pv WHERE pv.numero IN :numeros
Resultados:
id=3, numero=1, descripcion=Casa Central
id=4, numero=2, descripcion=Sucursal Norte
id=5, numero=5, descripcion=Sucursal Sur
id=6, numero=10, descripcion=Sucursal Centro
```

Nivel 3: Navegación de Entidades, JOINs y Subconsultas Simples

10. Navegación Implícita por Relaciones (Path Expressions)

- **Consigna:** Consultar todas las facturas de venta creadas por un usuario en particular navegando por su nombre de usuario de carga (usuarioCarga.usuario).

Consulta:



```java
private static void consulta10FacturasPorUsuarioCarga(EntityManager em, String usuario) {
        String jpql = "SELECT fv FROM FacturaVenta fv WHERE fv.usuarioCarga.usuario = :usuario";
        imprimirConsulta("10. Navegacion implicita por relaciones", jpql);

        List<FacturaVenta> facturas = em.createQuery(jpql, FacturaVenta.class)
                .setParameter("usuario", usuario)
                .getResultList();
        imprimirFacturas(facturas);
    }
```

Consola:

```
10. Navegacion implicita por relaciones
JPQL:
SELECT fv FROM FacturaVenta fv WHERE fv.usuarioCarga.usuario = :usuario
Resultados:
id=3, numero=1001, fechaEmision=2026-09-02, estado=EMITIDA, importeTotal=75000.0
id=4, numero=1002, fechaEmision=2026-09-03, estado=EMITIDA, importeTotal=25000.0
id=5, numero=1003, fechaEmision=2026-09-04, estado=EMITIDA, importeTotal=8000.0
id=6, numero=1004, fechaEmision=2026-09-05, estado=ANULADA, importeTotal=12500.0
id=7, numero=1005, fechaEmision=2026-09-06, estado=PENDIENTE, importeTotal=45000.0
id=8, numero=1006, fechaEmision=2026-09-07, estado=EMITIDA, importeTotal=90000.0
```

11. Cláusula INNER JOIN Explícita

- **Consigna:** Obtener todos los detalles de factura (FacturaVentaDetalle) que correspondan a facturas emitidas por un punto de venta determinado.

Consulta:

```java
private static void consulta11DetallesPorPuntoVenta(EntityManager em, int numeroPuntoVenta) {
        String jpql = """
                SELECT d
                FROM FacturaVentaDetalle d
                JOIN d.factura fv
                WHERE fv.puntoVenta.numero = :numeroPuntoVenta
                """;
        imprimirConsulta("11. INNER JOIN explicito", jpql);

        List<FacturaVentaDetalle> detalles = em.createQuery(jpql, FacturaVentaDetalle.class)
                .setParameter("numeroPuntoVenta", numeroPuntoVenta)
                .getResultList();
        imprimirDetalles(detalles);
    }
```

Consola:

```
11. INNER JOIN explicito
JPQL:
SELECT d
FROM FacturaVentaDetalle d
JOIN d.factura fv
WHERE fv.puntoVenta.numero = :numeroPuntoVenta
Resultados:
id=3, facturaNumero=1001, cantidad=1.0, precioUnitario=75000.0, importeSubtotal=75000.0
id=4, facturaNumero=1002, cantidad=2.0, precioUnitario=12500.0, importeSubtotal=25000.0
id=8, facturaNumero=1006, cantidad=1.0, precioUnitario=45000.0, importeSubtotal=45000.0
id=9, facturaNumero=1006, cantidad=2.0, precioUnitario=12500.0, importeSubtotal=25000.0
id=10, facturaNumero=1006, cantidad=5.0, precioUnitario=4000.0, importeSubtotal=20000.0
```

12. Cláusula LEFT JOIN (Inclusión de Nulos)

- **Consigna:** Listar la denominación de todos los artículos junto con la denominación de su marca asociada, incluyendo también aquellos artículos que no posean una marca asignada.

Consulta:

```java
private static void consulta12ArticulosConMarca(EntityManager em) {
        String jpql = """
                SELECT a.denominacion, m.denominacion
                FROM Articulo a
                LEFT JOIN a.marca m
                ORDER BY a.denominacion
                """;
        imprimirConsulta("12. LEFT JOIN", jpql);

        List<Object[]> filas = em.createQuery(jpql, Object[].class).getResultList();
        for (Object[] fila : filas) {
            System.out.println("articulo=" + fila[0] + ", marca=" + fila[1]);
        }
        imprimirSinResultados(filas);
    }
```

Consola:

```
12. LEFT JOIN
JPQL:
SELECT a.denominacion, m.denominacion
FROM Articulo a
LEFT JOIN a.marca m
ORDER BY a.denominacion
Resultados:
articulo=Cable HDMI, marca=Generica
articulo=Cuaderno A4, marca=null
articulo=Monitor 24 pulgadas, marca=Samsung
articulo=Mouse Inalambrico, marca=Logitech
articulo=Teclado Mecanico, marca=Logitech
```

13. Navegación Multinivel con JOINs Combinados

- **Consigna:** Obtener todas las facturas de venta que contengan al menos un detalle de artículo perteneciente a una marca específica.

Consulta:

```java
private static void consulta13FacturasPorMarca(EntityManager em, String denominacionMarca) {
        String jpql = """
                SELECT DISTINCT fv
                FROM FacturaVenta fv
                JOIN fv.detalles d
                JOIN d.listaPrecioArticulo lpa
                JOIN lpa.articulo a
                JOIN a.marca m
                WHERE m.denominacion = :denominacionMarca
                """;
        imprimirConsulta("13. Navegacion multinivel con JOINs combinados", jpql);

        List<FacturaVenta> facturas = em.createQuery(jpql, FacturaVenta.class)
                .setParameter("denominacionMarca", denominacionMarca)
                .getResultList();
        imprimirFacturas(facturas);
    }
```

Consola:

```
13. Navegacion multinivel con JOINs combinados
JPQL:
SELECT DISTINCT fv
FROM FacturaVenta fv
JOIN fv.detalles d
JOIN d.listaPrecioArticulo lpa
JOIN lpa.articulo a
JOIN a.marca m
WHERE m.denominacion = :denominacionMarca
Resultados:
id=3, numero=1001, fechaEmision=2026-09-02, estado=EMITIDA, importeTotal=75000.0
id=4, numero=1002, fechaEmision=2026-09-03, estado=EMITIDA, importeTotal=25000.0
id=6, numero=1004, fechaEmision=2026-09-05, estado=ANULADA, importeTotal=12500.0
id=8, numero=1006, fechaEmision=2026-09-07, estado=EMITIDA, importeTotal=90000.0
id=9, numero=1007, fechaEmision=2026-09-08, estado=EMITIDA, importeTotal=16500.0
```

14. Subconsulta en Cláusula WHERE

- **Consigna:** Listar las facturas de venta cuyo importeTotal sea estrictamente mayor al promedio de importeTotal de todas las facturas registradas.

Consulta:

```java
private static void consulta14FacturasMayoresAlPromedio(EntityManager em) {
        String jpql = """
                SELECT fv
                FROM FacturaVenta fv
                WHERE fv.importeTotal > (
                    SELECT AVG(fv2.importeTotal)
                    FROM FacturaVenta fv2
                )
                """;
        imprimirConsulta("14. Subconsulta en clausula WHERE", jpql);

        List<FacturaVenta> facturas = em.createQuery(jpql, FacturaVenta.class).getResultList();
        imprimirFacturas(facturas);
    }
```

Consola:

```
14. Subconsulta en clausula WHERE
JPQL:
SELECT fv
FROM FacturaVenta fv
WHERE fv.importeTotal > (
    SELECT AVG(fv2.importeTotal)
    FROM FacturaVenta fv2
)
Resultados:
id=3, numero=1001, fechaEmision=2026-09-02, estado=EMITIDA, importeTotal=75000.0
id=7, numero=1005, fechaEmision=2026-09-06, estado=PENDIENTE, importeTotal=45000.0
id=8, numero=1006, fechaEmision=2026-09-07, estado=EMITIDA, importeTotal=90000.0
```

Nivel 4: Agrupamiento (GROUP BY) y Filtros de Grupo (HAVING)

15. Agrupamiento Básico (GROUP BY)

- **Consigna:** Obtener la descripción del punto de venta, la cantidad de facturas emitidas por cada uno y la suma total facturada.

Consulta:

```java
private static void consulta15AgrupamientoPorPuntoVenta(EntityManager em) {
        String jpql = """
                SELECT pv.id, pv.descripcion, COUNT(fv), SUM(fv.importeTotal)
                FROM FacturaVenta fv
                JOIN fv.puntoVenta pv
                GROUP BY pv.id, pv.descripcion
                ORDER BY pv.descripcion
                """;
        imprimirConsulta("15. Agrupamiento basico GROUP BY", jpql);

        List<Object[]> filas = em.createQuery(jpql, Object[].class).getResultList();
        for (Object[] fila : filas) {
            System.out.println("puntoVentaId=" + fila[0] + ", puntoVenta=" + fila[1]
                    + ", cantidadFacturas=" + fila[2] + ", totalFacturado=" + fila[3]);
        }
        imprimirSinResultados(filas);
    }
```

Consola:

```
15. Agrupamiento basico GROUP BY
JPQL:
SELECT pv.id, pv.descripcion, COUNT(fv), SUM(fv.importeTotal)
FROM FacturaVenta fv
JOIN fv.puntoVenta pv
GROUP BY pv.id, pv.descripcion
ORDER BY pv.descripcion
Resultados:
puntoVentaId=3, puntoVenta=Casa Central, cantidadFacturas=2, totalFacturado=24500.0
puntoVentaId=6, puntoVenta=Sucursal Centro, cantidadFacturas=3, totalFacturado=190000.0
puntoVentaId=4, puntoVenta=Sucursal Norte, cantidadFacturas=1, totalFacturado=12500.0
puntoVentaId=5, puntoVenta=Sucursal Sur, cantidadFacturas=1, totalFacturado=45000.0
```

16. Agrupamiento con Condicional de Grupo (HAVING)

- **Consigna:** Obtener los nombres de los usuarios de carga que hayan registrado más de 5 facturas de venta en el sistema.

Consulta:

```java
private static void consulta16UsuariosConMasDeCincoFacturas(EntityManager em) {
        String jpql = """
                SELECT u.id, u.usuario, COUNT(fv)
                FROM FacturaVenta fv
                JOIN fv.usuarioCarga u
                GROUP BY u.id, u.usuario
                HAVING COUNT(fv) > :cantidadMinima
                """;
        imprimirConsulta("16. Agrupamiento con HAVING", jpql);

        List<Object[]> filas = em.createQuery(jpql, Object[].class)
                .setParameter("cantidadMinima", 5L)
                .getResultList();
        for (Object[] fila : filas) {
            System.out.println("usuarioId=" + fila[0] + ", usuario=" + fila[1] + ", cantidadFacturas=" + fila[2]);
        }
        imprimirSinResultados(filas);
    }
```

Consola:

```
16. Agrupamiento con HAVING
JPQL:
SELECT u.id, u.usuario, COUNT(fv)
FROM FacturaVenta fv
JOIN fv.usuarioCarga u
GROUP BY u.id, u.usuario
HAVING COUNT(fv) > :cantidadMinima
Resultados:
usuarioId=3, usuario=admin, cantidadFacturas=6
```

17. Agrupamiento y Agregación sobre Entidades Relacionadas

- **Consigna:** Obtener la denominación de cada marca, la cantidad total de unidades vendidas (SUM(cantidad)) y el subtotal acumulado, agrupado por marca.

Consulta:

```java
private static void consulta17VentasAgrupadasPorMarca(EntityManager em) {
        String jpql = """
                SELECT m.denominacion, SUM(d.cantidad), SUM(d.importeSubtotal)
                FROM FacturaVentaDetalle d
                JOIN d.listaPrecioArticulo lpa
                JOIN lpa.articulo a
                JOIN a.marca m
                GROUP BY m.denominacion
                ORDER BY m.denominacion
                """;
        imprimirConsulta("17. Agrupamiento sobre entidades relacionadas", jpql);

        List<Object[]> filas = em.createQuery(jpql, Object[].class).getResultList();
        for (Object[] fila : filas) {
            System.out.println("marca=" + fila[0] + ", unidadesVendidas=" + fila[1] + ", subtotalAcumulado=" + fila[2]);
        }
        imprimirSinResultados(filas);
    }
```

Consola:

```
17. Agrupamiento sobre entidades relacionadas
JPQL:
SELECT m.denominacion, SUM(d.cantidad), SUM(d.importeSubtotal)
FROM FacturaVentaDetalle d
JOIN d.listaPrecioArticulo lpa
JOIN lpa.articulo a
JOIN a.marca m
GROUP BY m.denominacion
ORDER BY m.denominacion
Resultados:
marca=Generica, unidadesVendidas=8.0, subtotalAcumulado=32000.0
marca=Logitech, unidadesVendidas=7.0, subtotalAcumulado=150000.0
marca=Samsung, unidadesVendidas=2.0, subtotalAcumulado=90000.0
```

Nivel 5: Subconsultas Correlacionadas, EXISTS, NOT EXISTS y Expresiones Condicionales

18. Subconsulta Correlacionada con EXISTS

- **Consigna**: Obtener la lista de todas las marcas que tienen al menos un artículo que haya sido facturado en alguna factura de venta.

Consulta:

```java
private static void consulta18MarcasConArticulosFacturados(EntityManager em) {
        String jpql = """
                SELECT m
                FROM Marca m
                WHERE EXISTS (
                    SELECT d.id
                    FROM FacturaVentaDetalle d
                    JOIN d.listaPrecioArticulo lpa
                    JOIN lpa.articulo a
                    WHERE a.marca = m
                )
                """;
        imprimirConsulta("18. Subconsulta correlacionada con EXISTS", jpql);

        List<Marca> marcas = em.createQuery(jpql, Marca.class).getResultList();
        imprimirMarcas(marcas);
    }
```

Consola:

```
18. Subconsulta correlacionada con EXISTS
JPQL:
SELECT m
FROM Marca m
WHERE EXISTS (
    SELECT d.id
    FROM FacturaVentaDetalle d
    JOIN d.listaPrecioArticulo lpa
    JOIN lpa.articulo a
    WHERE a.marca = m
)
Resultados:
id=1, codigo=1, denominacion=Logitech
id=2, codigo=2, denominacion=Samsung
id=3, codigo=3, denominacion=Generica
```

19. Subconsulta Correlacionada con NOT EXISTS

- **Consigna:** Obtener todos los artículos registrados en el sistema que **nunca** han sido incluidos en ningún detalle de factura de venta.

Consulta:

```java
private static void consulta19ArticulosNuncaFacturados(EntityManager em) {
        String jpql = """
                SELECT a
                FROM Articulo a
                WHERE NOT EXISTS (
                    SELECT d.id
                    FROM FacturaVentaDetalle d
                    JOIN d.listaPrecioArticulo lpa
                    WHERE lpa.articulo = a
                )
                """;
        imprimirConsulta("19. Subconsulta correlacionada con NOT EXISTS", jpql);

        List<Articulo> articulos = em.createQuery(jpql, Articulo.class).getResultList();
        imprimirArticulos(articulos);
    }
```

Consola:

```
19. Subconsulta correlacionada con NOT EXISTS
JPQL:
SELECT a
FROM Articulo a
WHERE NOT EXISTS (
    SELECT d.id
    FROM FacturaVentaDetalle d
    JOIN d.listaPrecioArticulo lpa
    WHERE lpa.articulo = a
)
Resultados:
id=7, codigo=ART05, denominacion=Cuaderno A4, rubro=Libreria, marca=null
```

20. Proyección Condicional (CASE WHEN)

- **Consigna:** Listar el número de factura, su importe total y una columna calculada llamada "Categoría" que clasifique la factura como:
  - "ALTO VALOR" si el importeTotal es mayor a $50,000.
  - "MEDIO VALOR" si el importeTotal está entre $10,000 y $50,000. ○ "BAJO VALOR" si el importeTotal es menor a $10,000. Ordenar los resultados de mayor a menor importe.

Consulta:



```java
private static void consulta20CategoriaFacturaPorImporte(EntityManager em) {
        String jpql = """
                SELECT fv.numero,
                       fv.importeTotal,
                       CASE
                           WHEN fv.importeTotal > :importeAlto THEN 'ALTO VALOR'
                           WHEN fv.importeTotal BETWEEN :importeMedioDesde AND :importeMedioHasta THEN 'MEDIO VALOR'
                           ELSE 'BAJO VALOR'
                       END
                FROM FacturaVenta fv
                ORDER BY fv.importeTotal DESC
                """;
        imprimirConsulta("20. Proyeccion condicional CASE WHEN", jpql);

        List<Object[]> filas = em.createQuery(jpql, Object[].class)
                .setParameter("importeAlto", 50000.0)
                .setParameter("importeMedioDesde", 10000.0)
                .setParameter("importeMedioHasta", 50000.0)
                .getResultList();
        for (Object[] fila : filas) {
            System.out.println("numero=" + fila[0] + ", importeTotal=" + fila[1] + ", categoria=" + fila[2]);
        }
        imprimirSinResultados(filas);
    }
```

Consola:

```
20. Proyeccion condicional CASE WHEN
JPQL:
SELECT fv.numero,
       fv.importeTotal,
       CASE
           WHEN fv.importeTotal > :importeAlto THEN 'ALTO VALOR'
           WHEN fv.importeTotal BETWEEN :importeMedioDesde AND :importeMedioHasta THEN 'MEDIO VALOR'
           ELSE 'BAJO VALOR'
       END
FROM FacturaVenta fv
ORDER BY fv.importeTotal DESC
Resultados:
numero=1006, importeTotal=90000.0, categoria=ALTO VALOR
numero=1001, importeTotal=75000.0, categoria=ALTO VALOR
numero=1005, importeTotal=45000.0, categoria=MEDIO VALOR
numero=1002, importeTotal=25000.0, categoria=MEDIO VALOR
numero=1007, importeTotal=16500.0, categoria=MEDIO VALOR
numero=1004, importeTotal=12500.0, categoria=MEDIO VALOR
numero=1003, importeTotal=8000.0, categoria=BAJO VALOR
```